

CloudShield AI

10 — LLM Integration Specification

Version 1.0 | Product-Level Engineering Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the implementation contract between CloudShield AI and external or local Large Language Model (LLM) providers, including provider adapters, request/response handling, authentication, context transfer, structured outputs, retries, safety controls, observability, cost management, testing, and operational governance.

Canonical principle: The LLM is an untrusted reasoning dependency. AI proposes. Deterministic systems validate. Governance decides. Cloud APIs execute. Evidence records. Trusted state is earned.

1. Purpose and Scope

This document specifies how CloudShield AI integrates with LLM providers without coupling product logic to a particular vendor, SDK, model family, or deployment topology.

- Define the provider-neutral LLM integration contract.
- Define hosted-provider and future local-model adapter responsibilities.
- Define request construction, structured output, error handling, retries, timeouts, and idempotency.
- Define model routing, capability discovery, configuration, quotas, cost controls, and observability.
- Define privacy, data minimization, credential handling, and provider-boundary security.
- Define test doubles and evaluation interfaces for deterministic CI and AI regression testing.
- Define how LLM responses become validated CloudShield AI intelligence or RemediationCandidate inputs.

This specification does not authorize direct cloud execution by an LLM. It also does not define foundation-model training or fine-tuning as an MVP requirement.

2. Relationship to the AI Architecture

Specification	Primary Responsibility
09 — Intelligence & AI Architecture	Defines why and where AI is used, context architecture, task taxonomy, validation, safety, and DAIL boundary.
10 — LLM Integration	Defines how CloudShield AI technically communicates with model providers.
11 — AI Safety & Guardrail Specification	Defines detailed safety/guardrail policy and adversarial controls.
DAIL specifications	Define deterministic candidate lifecycle, verification, promotion, evidence, and trusted-state rules.

The integration layer is subordinate to the Intelligence & AI Architecture. It must not introduce capabilities that bypass the safety and lifecycle boundaries defined there.

3. Integration Principles

- Provider-neutral contracts are mandatory.
- No provider SDK types leak into core domain models.
- All provider calls occur server-side through controlled adapters.
- Provider credentials are never exposed to browsers, users, model context, logs, or evidence exports.
- Model responses are untrusted until parsed and validated.
- Provider failures must produce explicit application states.
- Retries must be bounded, classified by error type, and safe against duplicate side effects.
- Material AI results retain provider/model/version metadata and correlation identifiers.
- Sensitive context is minimized before crossing the provider trust boundary.
- A provider switch must not alter authorization, validation, DAIL, or promotion rules.

4. LLM Integration Architecture

4.1 Logical components

Component	Responsibility
LLM Gateway	Single controlled integration boundary for model requests.
LLM Orchestrator	Maps CloudShield AI task to model route and manages execution.
Provider Registry	Stores approved provider/model capabilities and policies.
Provider Adapter	Translates common request contract into provider-specific API calls.
Request Builder	Builds provider-safe messages from validated task/context.
Response Parser	Parses provider response into provider-neutral response envelope.
Schema Validator	Validates structured output against registered schema.
Safety Validator	Applies integration-level safety checks and rejects unsafe responses.
Usage Meter	Captures tokens/usage/latency/cost metadata.
Retry Controller	Applies bounded, policy-driven retry behavior.
Circuit Breaker	Stops calls to unhealthy provider routes.
Trace/Evidence Writer	Records safe metadata and references for material requests/results.
Mock Adapter	Provides deterministic responses for automated tests.

4.2 Canonical flow

1. Product service submits an AI task to the LLM Gateway.
2. The gateway authenticates the internal caller and resolves tenant/scope.
3. The Orchestrator loads the approved task configuration and model route.
4. The Request Builder retrieves the already-authorized context and converts it into a provider-safe representation.
5. The Provider Adapter performs the outbound request.
6. The response is normalized into the provider-neutral LLMResponse envelope.
7. Structured parsing and schema validation run outside the model.
8. Safety and domain validation determine whether the response is usable.

9. Validated output is returned to the calling intelligence workflow and recorded as appropriate.
10. Infrastructure-affecting proposals are converted into explicit candidates and routed into DAIL; the LLM integration layer never promotes them.

5. Provider-Neutral Contract

5.1 LLMRequest

Field	Required	Description
request_id	Yes	Globally unique request identifier.
correlation_id	Yes	End-to-end trace identifier.
tenant_id	Yes	Tenant security context; must not be inferred by provider.
task_id	Yes	Registered CloudShield AI task.
task_version	Yes	Version of task behavior/prompt contract.
model_route	Yes	Approved logical model route, not a raw arbitrary provider model.
messages/context	Yes	Provider-neutral instruction and context representation.
output_schema	Conditional	Required for structured-output tasks.
safety_profile	Yes	Named safety/control profile.
timeout_ms	Yes	Maximum provider call duration.
budget	Conditional	Token/cost/usage limits.
metadata	No	Safe tracing metadata.

5.2 LLMResponse

Field	Description
request_id	Original request identity.
provider	Provider adapter identifier.
model	Actual model identifier/version returned by provider.
status	SUCCESS, REFUSED, INVALID, TIMEOUT, RATE_LIMITED, PROVIDER_ERROR, or similar.
content	Normalized model output.
structured_output	Parsed object when structured output is expected.
usage	Token/usage metrics when available.
latency_ms	Measured end-to-end provider latency.
finish_reason	Normalized provider completion status.
safety_flags	Integration safety signals.
raw_reference	Optional controlled reference to retained raw response; raw payload should not be logged by default.

5.3 Error contract

Error Class	Meaning	Retry
AUTHENTICATION_ERROR	Provider credential rejected	No automatic retry; alert/configuration action.
AUTHORIZATION_ERROR	Provider access not permitted	No automatic retry.
RATE_LIMITED	Provider quota/throttle	Bounded backoff if task remains valid.
TIMEOUT	Provider exceeded deadline	Limited retry for idempotent

Error Class	Meaning	Retry
		request.
TRANSIENT_PROVIDER_ERROR	Temporary upstream failure	Bounded exponential backoff.
INVALID_REQUEST	CloudShield request violates provider contract	No retry; fix caller/config.
CONTEXT_TOO_LARGE	Input exceeds route limit	Rebuild/compress context deterministically.
CONTENT_REJECTED	Provider safety policy refusal	No blind retry with weakened controls.
PARSER_ERROR	Response cannot be normalized	Limited retry only when provider behavior is known transient; otherwise fail.
UNKNOWN	Unclassified failure	Fail safely; preserve diagnostics and alert if repeated.

6. Provider Adapter Requirements

- Implement the common LLMRequest/LLMResponse/Error contract.
- Translate provider-specific authentication, message formats, schemas, tool interfaces, and usage reporting.
- Normalize provider error codes into CloudShield AI error classes.
- Never return provider SDK objects to domain services.
- Apply provider-specific limits without changing domain semantics.
- Support request timeout and cancellation.
- Expose provider/model capability metadata.
- Support safe redaction of provider request/response diagnostics.
- Be independently testable using mocked provider responses.

7. Hosted Provider Integration

MVP should use one approved hosted LLM provider through a Provider Adapter. The exact provider may be selected by deployment configuration; core CloudShield AI code must remain provider-agnostic.

Control	Requirement
Credential storage	Secret manager or secure runtime injection.
Network	TLS for all provider traffic.
Endpoint	Allowlisted provider endpoint/configuration.
Data policy	Approved provider terms and retention configuration for intended data class.
Model allowlist	Only models approved for each task/data sensitivity class.
Timeout	Per-task bounded deadline.
Quota	Global and tenant/task usage limits.
Telemetry	Usage, latency, error, and model version metadata.
Fallback	Only to pre-approved routes with equivalent safety/data policy.

8. Future Local / Private Model Integration

CloudShield AI may later integrate a private or self-hosted open-weight model through the same adapter contract.

Requirement	Local Integration Rule
Network boundary	Private/internal network where practical.
Authentication	Workload identity/service credentials.
Model lifecycle	Explicit model artifact/version management.
Inference capacity	Queueing and concurrency controls.
Privacy	Data remains within approved deployment boundary.
Observability	Expose normalized usage/latency/error metadata.
Safety	Same external validation and DAIL controls as hosted models.
Rollback	Model route can be reverted without changing task semantics.

A local model is an infrastructure/provider choice, not a relaxation of CloudShield AI's safety model.

9. Authentication and Credential Management

- Provider API keys or credentials are stored outside source control.
- Credentials are injected only into the server-side adapter/runtime that requires them.
- Credentials are never included in prompts, user-visible responses, client bundles, or standard application logs.
- Rotate credentials according to provider and organizational security policy.
- Separate credentials by environment where practical.
- Use distinct identities for development, staging, and production.
- Revoke compromised credentials immediately and invalidate affected provider routes.
- Audit administrative changes to provider credentials and model configuration.

10. Request Construction

10.1 Request pipeline

11. Resolve registered task configuration.
12. Resolve approved model route.
13. Resolve authorized tenant/scope.
14. Build deterministic context.
15. Apply data classification and redaction.
16. Apply context-size/relevance limits.
17. Load versioned instructions and output schema.
18. Construct provider-neutral request.
19. Validate request before sending.
20. Send through the selected adapter.

10.2 Request invariants

- Every request has a task ID and correlation ID.
- Every protected request has a tenant/scope context.
- Every material structured-output request specifies an expected schema.
- The model route is selected from an allowlist.
- The request cannot contain unrestricted cloud credentials.
- Context must not contain data outside caller authorization.
- Task-specific safety profile must be present.

- Request size and estimated cost must remain within configured limits.

11. Context Transfer and Data Minimization

Data	Default Treatment
Cloud credentials	Never send.
Secrets/API tokens	Never send unless a future task explicitly proves necessity and has dedicated controls; MVP excludes them.
Asset metadata	Send only relevant authorized fields.
Configuration	Send minimized relevant configuration/diff.
Findings	Send relevant finding/rule/evidence references and facts.
Dependency graph	Send bounded relevant subgraph with freshness/provenance.
Policies	Send applicable policy text/structured constraints.
User input	Send as untrusted task content; cannot override system policy.
Evidence artifacts	Prefer summaries/references; retrieve raw artifacts only when required and authorized.

Before a request crosses the provider boundary, CloudShield AI should know what data classes it is sending, why they are required, and which tenant/scope owns them.

12. Structured Output Integration

- Prefer provider-supported structured-output mechanisms when available.
- Treat provider-side schema enforcement as an additional control, not the final validator.
- Perform application-side schema validation after receiving the response.
- Reject unknown or prohibited fields for security-sensitive tasks.
- Validate identifiers against CloudShield AI canonical objects.
- Validate numeric ranges, enums, resource types, and target scope.
- Record schema version with material AI results.
- Never interpret a syntactically valid response as semantically safe without domain validation.

13. Tool / Function Calling Integration

If tool calling is introduced, it must be represented as a request from the model to the Tool Broker rather than a direct provider-to-cloud execution path.

Step	Control
Model emits tool request	Parse and validate tool name/arguments.
Broker receives request	Authorize independently using caller/tenant context.
Risk evaluation	Apply tool risk class and policy.
Execution	Run with least-privilege identity and bounded scope.
Result	Sanitize and return only necessary data.
Audit	Record request, authorization, execution, and outcome.
High-impact action	Require explicit human/policy approval and DAIL lifecycle where applicable.

MVP should favor read-only tools and avoid arbitrary infrastructure mutation through tool calling.

14. Retry, Timeout, and Circuit-Breaker Policy

14.1 Retry policy

- Retry only classified transient failures.
- Use bounded exponential backoff with jitter.
- Do not retry authentication, authorization, malformed-request, or policy-refusal failures automatically.
- Respect provider retry-after signals where available.
- Stop retrying when the task deadline is exceeded.
- Record retry count and final outcome.

14.2 Timeout policy

- Every provider request has an explicit timeout.
- Interactive tasks use shorter deadlines than background investigations.
- A timeout never implies success.
- Timed-out requests are safe to retry only when duplicate invocation has no harmful side effect; model inference itself is treated as non-authoritative.
- The application must distinguish provider timeout from downstream candidate execution state.

14.3 Circuit breaker

Repeated provider failures should open a circuit for the affected route, preventing cascading resource exhaustion. Recovery probes should be bounded and must not bypass normal safety controls.

15. Idempotency and Request Deduplication

- Every LLM request has a unique request ID.
- Asynchronous AI jobs should use an idempotency key tied to task, input/context version, and intended operation.
- Duplicate responses must not create duplicate candidates or duplicate governance actions.
- A model retry may create a new provider request ID while retaining the same logical AI job ID.
- Candidate creation remains protected by domain-level uniqueness and DAIL rules.

16. Model Routing

Route Attribute	Requirement
Logical route	Application refers to a named route such as SECURITY_EXPLAINER or REMEDIATION_ADVISOR.
Provider mapping	Configuration maps logical route to approved provider/model.
Data policy	Route defines permitted data classification.
Task capability	Route declares supported schema/context/tool capabilities.
Reliability tier	Route declares timeout/retry/fallback policy.
Cost tier	Route declares budget expectations.
Version	Model version changes are configuration events

Route Attribute	Requirement
	requiring evaluation.

Domain services should request logical routes, not hard-code vendor model identifiers.

17. Usage, Cost, and Quotas

- Track usage by tenant, task, model route, environment, and time period where appropriate.
- Apply global and per-tenant quotas to prevent runaway AI spend.
- Set task-level maximum context/output sizes.
- Reject or queue work when configured budgets are exhausted.
- Monitor cost anomalies and unexpected token growth.
- Prefer deterministic answers when AI adds little value.
- Cache safe reusable context/results only when version and authorization conditions remain valid.

18. Observability

Metric	Purpose
request_count	Volume by route/task.
success_rate	Provider/application reliability.
error_rate	Operational health.
timeout_rate	Latency/availability signal.
rate_limit_count	Provider quota pressure.
validation_failure_rate	AI output quality/safety signal.
latency_p50/p95/p99	Performance.
input/output_tokens	Usage/cost.
estimated_cost	Budget monitoring.
fallback_rate	Provider/model reliability.
safety_rejection_rate	Security signal.
duplicate_request_rate	Idempotency/queue health.

Logs must contain correlation IDs and safe metadata while excluding raw credentials and unnecessary sensitive prompts/responses.

19. Privacy and Provider Boundary

- Classify every context element before transmission.
- Use the minimum data required for the task.
- Review provider retention/training/data-use settings before production use.
- Do not send secrets, authentication tokens, or unnecessary personal information.
- Respect tenant-specific data residency/provider restrictions when applicable.
- Document what data crosses the external provider boundary.
- Provide configuration to disable AI features when policy requires it.
- Retain sufficient metadata for audit without retaining sensitive raw payloads by default.

20. Security Threats at the LLM Boundary

Threat	Integration Mitigation
--------	------------------------

Threat	Integration Mitigation
Credential theft	Server-side secret storage; never prompt-visible.
Prompt injection	Context labeling, safety profiles, output validation.
Indirect injection	Treat retrieved content as untrusted data.
Provider compromise	Provider allowlist + deterministic post-validation.
Cross-tenant leakage	Authorization before context assembly; tenant-aware traces/caches.
Request tampering	TLS + internal authenticated service calls.
Response tampering	TLS + schema/domain validation.
Denial of service	Quotas, timeouts, queue limits, circuit breakers.
Cost abuse	Budgets, quotas, task limits.
Model substitution	Approved route/model allowlist + configuration audit.

21. LLM Response Lifecycle

State	Meaning	Next States
REQUESTED	Logical AI task accepted.	DISPATCHED, REJECTED
DISPATCHED	Provider request initiated.	COMPLETED, FAILED, TIMED_OUT
COMPLETED	Provider returned a response.	VALIDATING
VALIDATING	Schema/domain/safety validation running.	VALIDATED, INVALID
VALIDATED	Response accepted for intended advisory/candidate workflow.	CONSUMED
INVALID	Response rejected by validator.	RETRYED, FAILED
FAILED	Request cannot complete safely.	RETRYED, TERMINAL
TIMED_OUT	Provider exceeded deadline.	RETRYED, TERMINAL
CONSUMED	Validated result handed to product workflow.	FINAL

The LLM lifecycle is separate from the RemediationCandidate and DAIL lifecycle. A COMPLETED or VALIDATED model response is never equivalent to VERIFIED or TRUSTED infrastructure state.

22. AI-to-Candidate Integration Contract

Requirement	Rule
Target identity	Must reference canonical stable asset/resource identity.
Scope	Must match authorized tenant/account/region scope.
Change format	Must use registered structured change schema.
Evidence	Recommendation must include relevant source/evidence references where required.
Uncertainty	Missing information must be represented explicitly.
Policy	Candidate must be checked against deterministic policy.
Versioning	AI request/context/output versions recorded.
DAIL handoff	Candidate enters DAIL lifecycle before verification/promotion.

23. Evaluation and Testing

23.1 Integration test layers

- Adapter contract tests using mocked provider responses.
- Schema parsing and malformed-response tests.
- Provider error normalization tests.
- Timeout/retry/circuit-breaker tests.
- Credential/configuration failure tests.
- Context redaction and tenant-isolation tests.
- Prompt injection and indirect-injection integration tests.
- Model version regression tests.
- Candidate-generation validation tests.
- Load and quota tests.

23.2 Golden provider fixtures

Maintain versioned response fixtures for representative provider behavior: successful structured responses, refusals, truncation, malformed JSON, rate limits, transient errors, timeouts, and unexpected fields. Fixtures enable deterministic CI without requiring a live provider call.

24. Local Mock and Test Adapter

Capability	Mock Behavior
Structured response	Return deterministic registered fixture.
Latency	Configurable simulated delay.
Failure injection	Simulate timeout, rate limit, provider error, malformed output.
Usage	Return deterministic token/cost metadata.
Model metadata	Expose stable test model ID/version.
Replay	Replay recorded safe test fixtures.
Security testing	Generate adversarial outputs without external provider calls.

The MockLLMAdapter is required for unit and integration testing of the CloudShield AI intelligence and DAIL integration paths.

25. Deployment and Environment Configuration

Environment	LLM Policy
Development	Mock adapter by default; hosted adapter optional with restricted test credentials.
Test/CI	Mock/evaluation adapters; live provider calls minimized and isolated.
Staging	Approved provider route with non-production data only.
Production	Approved provider/model routes, production data policy, quotas, monitoring, and credential controls.

- Provider credentials and model route configuration are environment-specific.

- Production configuration changes require review and audit.
- A production deployment must fail safe if no approved model route is configured for a required task.
- Provider endpoint allowlists prevent accidental routing to unapproved destinations.

26. Disaster Recovery and Provider Failure

- If AI is unavailable, deterministic security inventory and rule-based analysis must remain available where possible.
- AI-dependent workflows should enter explicit degraded states rather than fabricate results.
- Approved fallback models/routes may be used only when they meet the task's data and safety policy.
- Pending AI jobs should be resumable or safely re-created using logical job IDs.
- Provider recovery must not automatically replay infrastructure changes; DAIL state remains authoritative.
- Operational dashboards should expose provider outage and degraded AI capability.

27. Change Management

21. Register or modify provider/model route configuration.
22. Validate capabilities, security/data policy, schemas, and limits.
23. Run adapter contract tests.
24. Run golden and adversarial AI evaluations.
25. Compare quality, latency, cost, and safety metrics against the current route.
26. Approve the change.
27. Deploy through controlled configuration rollout.
28. Monitor production telemetry.
29. Rollback route/model configuration if safety or reliability thresholds regress.

28. MVP Integration Baseline

- One hosted provider adapter behind a provider-neutral interface.
- MockLLMAdapter for deterministic CI.
- Logical model routes rather than provider model IDs in domain code.
- Server-side provider credentials.
- Structured outputs for candidate-generating tasks.
- Application-side schema and domain validation.
- Tenant-aware context construction before provider calls.
- No unrestricted tool calling or direct cloud mutation from the LLM.
- Bounded timeouts/retries and provider circuit breaker.
- Usage/cost telemetry and quotas.
- Versioned task/prompt/schema/model configuration.
- Safe degraded behavior when provider is unavailable.
- Integration with RemediationCandidate and DAIL rather than direct promotion.

29. Future Integration Capabilities

- Multiple hosted providers with policy-based routing.
- Private/self-hosted open-weight model adapter.
- Model ensembles or reviewer models for high-risk reasoning.

- Controlled read-only tool use.
- Private retrieval and model-serving infrastructure.
- Provider-independent structured tool calling.
- Advanced semantic caching with context/version invalidation.
- Automated model evaluation gates in CI/CD.
- Fine-tuning/domain adaptation where measurable gains justify operational complexity.

30. Security Acceptance Criteria

30. No provider credential is accessible to the browser or model context.
31. No domain service directly imports a provider SDK.
32. All production model routes are allowlisted and versioned.
33. All structured candidate-generating responses are validated outside the model.
34. Provider errors cannot bypass authorization, policy, DAIL, or promotion gates.
35. Cross-tenant context cannot reach a provider through a tested application path.
36. Rate limits, quotas, timeouts, and circuit breakers prevent unbounded provider usage.
37. Prompt injection and malicious retrieved-content fixtures are included in integration tests.
38. Provider/model changes require evaluation and controlled rollout.
39. Provider outage produces explicit degraded behavior rather than fabricated or trusted state.

31. Functional Acceptance Criteria

40. A product task can invoke an approved logical model route without knowing the provider implementation.
41. The adapter returns a normalized response regardless of provider-specific response format.
42. Structured output can be parsed and validated against a registered schema.
43. A provider rate limit produces bounded retry behavior.
44. A provider timeout produces an explicit timeout state.
45. A malformed response cannot create a RemediationCandidate.
46. A valid candidate-generating response can be handed to DAIL with exact provenance.
47. A model route can be replaced through configuration without changing domain logic.
48. The MockLLMAdapter can exercise success and failure paths deterministically.

32. Traceability

Source	Integration Dependency
00 — Master System Specification	Product intelligence role, governance, DAIL positioning.
01 — Software Requirements Specification	AI abstraction layer, provider flexibility, structured outputs, security and MVP boundaries.
02 — System Architecture Specification	AI service boundary, asynchronous workers, provider adapters, data flow.
03 — Functional Specification	AI explanation, remediation, candidate, verification and promotion workflows.
04 — Product & UX Specification	AI output presentation, uncertainty, safe-path UX.
05 — Domain & Data Model	AIRequest, AIResponse, AIContext, AIRecommendation, Candidate, Evidence.
06 — State & Lifecycle Specification	Version binding, invalidation, reconciliation, trusted-state rules.
07 — Identity & Dependency Specification	Stable identifiers, dependency provenance, impact context.

Source	Integration Dependency
08 — Security & Threat Model	Credential, tenant, AI, network, supply-chain and fail-closed controls.
09 — Intelligence & AI Architecture	Task taxonomy, context engineering, model abstraction, validation, evaluation, DAIL boundary.

33. Engineering Decisions

Decision	Rationale
Provider-neutral integration	Avoid vendor lock-in and protect domain architecture.
One hosted provider for MVP	Reduces operational complexity while preserving adapter flexibility.
Mock adapter required	Makes AI workflows deterministic and testable.
Logical model routes	Allow model/provider changes without domain code changes.
Structured outputs	Enable deterministic validation and candidate conversion.
No direct cloud tool authority	Preserves security and DAIL lifecycle boundary.
Server-side credentials only	Reduces credential exposure.
Deterministic context assembly	Improves grounding, authorization, and reproducibility.
Bounded retries/circuit breaker	Prevents provider instability from cascading through the product.

34. Definition of Done

- Provider-neutral LLMRequest/LLMResponse/Error contracts are implemented.
- At least one hosted ProviderAdapter is implemented and tested.
- MockLLMAdapter supports success and failure fixtures.
- Logical model routes are configurable.
- Credentials are stored and injected securely.
- Tenant/scope authorization occurs before context transmission.
- Sensitive data is minimized/redacted before provider calls.
- Structured output schemas are versioned and validated outside the provider.
- Retry, timeout, rate-limit, and circuit-breaker behavior is tested.
- Usage, latency, errors, and validation outcomes are observable.
- Candidate-generating outputs integrate with RemediationCandidate/DAIL.
- Provider/model changes have an evaluation and rollback path.
- Production behavior is safe when the provider is unavailable.

35. Glossary

Term	Definition
LLM	Large Language Model used by CloudShield AI for bounded reasoning and language tasks.
LLM Gateway	Controlled internal boundary through which CloudShield AI accesses models.
Provider Adapter	Implementation translating provider-neutral requests into provider-specific API calls.
Logical Model Route	Named CloudShield AI configuration representing an

Term	Definition
	approved model capability.
LLMRequest	Provider-neutral request contract.
LLMResponse	Provider-neutral normalized response contract.
Model Route	Approved mapping of task requirements to a provider/model configuration.
Structured Output	Machine-readable model response conforming to a registered schema.
MockLLMAdapter	Deterministic adapter used for testing and evaluation.
Tool Broker	Independent authorization boundary for any future model-requested tool execution.
RemediationCandidate	Explicit proposed infrastructure change subject to deterministic lifecycle controls.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem controlling verification, governance, promotion, and trusted-state transitions.